Assignment 2.1: Programming Task 1

Course: Programming for Artificial Intelligence

Assignment: NumCompute Toolkit

Submission Type: Group Report

Team Members:

Risat Rahaman (ID - a1988117)
Saadman Sakib (ID - a3201424)
Benzamin Yasir (ID - a1984262)
Mazharul Islam Rakib (ID - a1990942)

University: Adelaide University
Due Date: 01/05/2026

Abstract

NumCompute is a self-contained Python package that re-implements the core scikit-learn workflow using only NumPy: CSV ingestion, preprocessing transformers, sorting and search primitives, ranking and percentiles, descriptive statistics, classification and regression metrics, finite-difference optimisation, and a chained Pipeline API. Every public function is fully vectorised, validated, and documented with shapes and complexity. On a 100,000-element float64 workload our vectorised mean is 164 times faster than its Python-loop equivalent, and our central-difference gradient achieves a max absolute error of 4.3×10^{-10} against the analytic solution. The library passes a test suite covering edge cases such as empty arrays, all-NaN inputs, tied values, and zero-variance features.

1. Project Overview

NumCompute exposes scikit-learn-style abstractions over NumPy primitives. The package is organised by responsibility: data I/O, preprocessing, sorting and selection, ranking, descriptive statistics, evaluation metrics, finite-difference optimisation, and a chained Transformer/Estimator/Pipeline API. The full surface area is summarised in Table 1.

Package Structure

Module	Key Components	Author
io.py	load_csv (genfromtxt wrapper, NaN fill), get_column	Risat Rahaman
preprocessing.py	StandardScaler, MinMaxScaler, OneHotEncoder, Simple Imputer	Risat Rahaman
sort_search.py	stable_sort, multi_key_sort, topk, quickselect, binary_search	Saadman Sakib
rank.py	rank (average/dense/ordinal), percentile (4 modes)	Saadman Sakib
stats.py	mean/var/std/min/max/median, histogram, quantile, Welford	Benzamin Yasir
metrics.py	accuracy, precision, recall, F1, confusion_matrix, ROC/AUC, MSE	Benzamin Yasir
optim.py	grad (forward/central FD), jacobian	M. I. Rakib
pipeline.py	Transformer, Estimator, Pipeline (chained fit/transform/predict)	M. I. Rakib
utils.py	distances, ReLU, sigmoid, softmax, logsumexp, batching	Risat Rahaman
benchmarking.py	time_function, Benchmark class, loop-vs-vectorised harness	Team

Table 1. NumCompute module breakdown with key components and responsible author.

2. Design Choices

2.1 Vectorisation Strategy

All hot-path computations delegate entirely to NumPy ufuncs and array operations, eliminating per-element Python overhead. Examples:

- StandardScaler.transform — single broadcast subtraction and division across the whole matrix.
- OneHotEncoder.transform — outer equality broadcast (n, 1) == (1, k) produces the full binary matrix in one expression.
- topk — uses np.argpartition for O(n) selection, then argsort only on the k results.
- rank (average method) — uses np.bincount with weights to accumulate rank sums per tie-group without any loop.
- confusion_matrix — vectorised label-to-index lookup via a dictionary comprehension and NumPy fancy indexing.
- logsumexp — max-shifted sum prevents overflow: log(sum(exp(xi))) = m + log(sum(exp(xi - m))).

2.2 Numerical Stability

Stability is enforced via well-known stable forms rather than ad-hoc clipping:

- Softmax / logsumexp: max-shift applied before exponentiation; tested with inputs up to +/-700.
- StandardScaler: zero-variance columns have their std forced to 1.0, avoiding division by zero.
- MinMaxScaler: constant columns get max = min + 1.0 so the denominator is never zero.
- Sigmoid: input clipped to [-500, 500] before exp to prevent inf.
- _safe_divide (metrics): returns a configurable zero_division sentinel when denominator is zero.
- Welford's online algorithm in stats.py computes mean and M_2 in a single pass with the update $\mu_n = \mu_{n-1} + (x_n - \mu_{n-1}) / n$, avoiding catastrophic cancellation versus the naïve $\Sigma x^2 - (\Sigma x)^2/n$ form.
- ROC curve: inserts (0,0) and (1,1) anchor points; AUC uses the trapezoidal rule via np.diff
- NaN handling: all stats functions expose skipna=True; rank/percentile strip NaNs before sorting.

2.3 API Consistency

Every public function validates inputs at entry: empty arrays, mismatched shapes, out-of-range axes, unrecognised method strings, and non-1D inputs to 1D-only routines all raise descriptive ValueError or TypeError messages. Axis semantics follow NumPy conventions (None = global, integer = per-axis). The Pipeline class detects whether a step accepts y via inspect.signature, so unsupervised transformers and supervised estimators can be chained without special-casing.

2.4 Pipeline Architecture

The Pipeline class accepts a list of (name, step) tuples and routes data sequentially. During fit, intermediate steps that expose transform() are applied in-place so the next step always receives transformed data. The helper _accepts_y() uses inspect.signature to avoid passing y to unsupervised transformers. The final step may be an Estimator that only implements predict(), not transform().

3. Algorithmic Highlights

3.1 Tie-aware Average Ranking in O(n log n) without Python loop

Naïve rank averaging with ties typically uses an inner loop over tie-groups. We avoid this by exploiting the fact that np.bincount is a fully vectorised group reducer when group labels are consecutive integers. The procedure: (i) stable argsort the data, (ii) detect group boundaries by comparing adjacent sorted values, (iii) use the boundary mask to assign group labels, (iv) bincount with weights to compute ranks per group, (v) divide by per-group counts. Steps (ii) (v) are entirely loop-free.

3.2 Top-k Selection: O(n) instead of O(n log n)

topk uses np.argpartition (introselect) to obtain an unordered set of the k extremal indices in linear time, then sorts only those k items. For k = 10 against a 1M element array we measure 3.6 ms, roughly 33 times faster than a full np.sort (10.2 ms) confirming the asymptotic advantage. quickselect is provided as a pure Python educational fallback. It is linear on average but around 450x slower than the NumPy alternative due to interpreter overhead.

3.3 Finite-Difference Gradient Order

Central differences have truncation error $O(h^2)$ versus $O(h)$ for forward differences. With $h = 10^{-5}$, central differences attain ~25,000× better accuracy at twice the cost

4. Testing & Edge Cases

Tests are grouped by module and cover both nominal behaviour and adversarial edge cases including empty arrays, all-equal values, duplicate/tied data, extreme k values, NaN inputs, and non-contiguous strides.

Module	Scenario	Assertion
io	Empty CSV file	Returns ([], [])
io	CSV with blank cells	Blanks => np.nan; float cast succeeds
preprocessing	StandardScaler - zero-variance col	std forced to 1.0; no ZeroDivision
preprocessing	MinMaxScaler - constant feature	max = min+1; output = feature_range[0]
preprocessing	OneHotEncoder - unseen category	Produces all-zero row; no exception
sort_search	stable_sort 0-d array	Returns scalar copy unchanged
sort_search	topk k=1 and k=n	k=1 => global max/min; k=n => full sort
sort_search	quickselect k=0 (minimum)	Returns np.min(arr)
sort_search	binary_search on duplicate values	Returns leftmost index; found=True

sort_search	binary_search - value not present	found=False; index = insertion point
rank	All-equal values - average method	All ranks = (n+1)/2
rank	rank - dense method with ties	Max rank = no. of distinct values
rank	percentile q=0 and q=100	Returns min and max of data
rank	percentile - array with NaNs	NaNs skipped; matches np.nanpercentile
stats	Welford - single element	sample_variance = NaN; variance = 0
stats	histogram - all-NaN input	Raises ValueError (no valid values)
metrics	accuracy - all correct	Returns 1.0
metrics	precision/recall - zero TP	Returns zero_division sentinel (0.0)
metrics	roc_curve - single-class input	Raises ValueError
optim	grad central vs forward - quadratic	Central error < forward error
optim	jacobian shape (m, n)	Shape = (output_dim, input_dim)
pipeline	Pipeline - no steps	Raises ValueError
utils	logsumexp - large positive values	No overflow; matches scipy reference
utils	batch - arrays of different lengths	Raises ValueError

Table 2. Unit test registry covering all NumCompute modules.

Duplicate-heavy arrays (all same value) validate tie-handling in rank(), topk(), and sort(). Extreme k values (k=1, k=n) are checked in both topk and quickselect.

5. Benchmark Results

All benchmarks were executed on a single core (no parallelism) using time.perf_counter with n_repeat=7 and one warm-up iteration. Table 3 shows some important examples of the comparisons.

5.1 Loop vs. Vectorised (n = 100,000 samples)

Operation	Python Loop (mean)	Vectorised (mean)	Speedup
Mean	~85.81 ms	~524 us	163.7x
Accuracy	~187.58 ms	~2.29 ms	81.8x
Sum-of-Squared Diff	~193.39 ms	~23.90 ms	8.1x
Stable_sort	~110.91 ms	~65.03 us	1705.6x
Binary search	~2.66 ms	~1.53 us	1732.9x

Table 3. Median wall-clock times from the Benchmark harness (benchmarking.py).

6. Reflections on Teamwork

6.1 Division of Workload

Risat completed io, preprocessing and utils; Saadman did sorting, search, and ranking; Rakib owned optimisation and the pipeline API. Benzamin completed stats and metrics. Everyone had an equal contribution to benchmarking.

6.2 Ensuring Consistency

We agreed on a shared coding standard early, where every function validates inputs, uses np.asarray and raises errors and exceptions. This helped us with integration of all our codes. The pipeline module also connected the other modules together.

6.3 Handling numerical errors

Our discussions were on edge cases: how to handle constant features in scalers, tie-breaking in sort and rank, and NaN propagation semantics. Resolving these explicitly in code (rather than documentation alone) improved overall reliability.

6.4 Testing Workflow

Each of us wrote unit tests alongside their module. A shared test review session before submission caught several off-by-one errors in rank() and an incorrect confusion-matrix index in an early version of metrics.py.